

Week_10_Li_Code_file_ipynb_final_version

May 10, 2026

1 Import libs

```
[5]: # import tensorflow and keras
import tensorflow as tf
from tensorflow import keras
from keras import layers
import keras.backend as K

# import numpy to create arrays for trial runs
import numpy as np
# matplotlib for visualisations
import matplotlib.pyplot as plt
# if using Colab, for later saving of models and loading data
from google.colab import drive
drive.mount('/content/drive')
from pathlib import Path
from PIL import Image
from itertools import product
import pandas as pd
```

Mounted at /content/drive

```
[ ]: #!unzip -q /content/drive/MyDrive/BMET5933/WEEK_10/hep2imgcnn.zip -d /content/
    ↪drive/MyDrive/BMET5933/WEEK_10
```

```
[ ]: # dataset path config
train_vali_dataset_path = Path("/content/drive/MyDrive/BMET5933/WEEK_10/
    ↪hep2imgcnn")

# get class names and create mapping to labels
class_names = sorted([folder.name for folder in train_vali_dataset_path.
    ↪iterdir() if folder.is_dir()])
class_to_label = {class_name: idx for idx, class_name in enumerate(class_names)}
print("Class to label mapping:", class_to_label)
```

Class to label mapping: {'centromere': 0, 'coarse_speckled': 1, 'fine_speckled': 2, 'homogeneous': 3, 'nucleolar': 4}

AI acknowledge: Chatgpt5.5 used here for learning and suggesting how to design training/validation/test(later) dataset.

```
[ ]: SEED=42
BATCH_SIZE=32
IMAGE_DIR=str(train_vali_dataset_path) # this is the path to the directory
↳containing the class subdirectories
RESCALED_IMAGE_SIZE=(51,51) # images will be all resized to this - they will
↳have 3 channels

training_ds = keras.preprocessing.image_dataset_from_directory(
    IMAGE_DIR,
    batch_size=BATCH_SIZE,
    image_size=RESCALED_IMAGE_SIZE,
        shuffle=True,
        seed = SEED,
    validation_split=0.3,
    subset='training',
        color_mode = 'grayscale'
)

validation_ds = keras.preprocessing.image_dataset_from_directory(
    IMAGE_DIR,
    labels='inferred',
    label_mode='int',
    batch_size=BATCH_SIZE,
    image_size=RESCALED_IMAGE_SIZE,
    shuffle=True,
    seed = SEED,
    validation_split=0.3,
    subset='validation',
    color_mode = 'grayscale'
)

# you can optimise data loading with prefetching
PREFETCH_SIZE = tf.data.AUTOTUNE
training_ds = training_ds.prefetch(buffer_size=PREFETCH_SIZE)
validation_ds = validation_ds.prefetch(buffer_size=PREFETCH_SIZE)
```

Found 428 files belonging to 5 classes.
Using 300 files for training.
Found 428 files belonging to 5 classes.
Using 128 files for validation.

```
[ ]: # define a shallow CNN model
def shallow_model(input_shape, num_classes, filter_size=5, strides=1,
    ↳padding='same', num_filters=16):
```

```

        model = keras.Sequential([
            layers.Input(shape=input_shape),
            layers.Conv2D(num_filters, kernel_size=(filter_size,
↪filter_size), activation='relu', strides=(strides, strides),
↪padding=padding),
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Flatten(),
            layers.Dense(num_classes, activation='softmax'),

        ])
        return model

# initialise model and print summary
shallow_cnn = shallow_model(input_shape=(51, 51, 1),
↪num_classes=len(class_names))

# check model architecture
shallow_cnn.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 51, 51, 16)	416
max_pooling2d (MaxPooling2D)	(None, 25, 25, 16)	0
flatten (Flatten)	(None, 10000)	0
dense (Dense)	(None, 5)	50,005

Total params: 50,421 (196.96 KB)

Trainable params: 50,421 (196.96 KB)

Non-trainable params: 0 (0.00 B)

```

[ ]: # define a deeper CNN
def deeper_model(input_shape, num_classes, filter_size=[3, 3, 3], strides=[1,
↪1, 1], padding = 'same'):
    model = keras.Sequential([
        layers.Input(shape=input_shape),

```

```

        layers.Conv2D(32, kernel_size=(filter_size[0], filter_size[0]),
↪activation='relu', strides=(strides[0], strides[0]), padding=padding),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(filter_size[1], filter_size[1]),
↪activation='relu', strides=(strides[1], strides[1]), padding=padding),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(128, kernel_size=(filter_size[2],
↪filter_size[2]), activation='relu', strides=(strides[2], strides[2]),
↪padding=padding),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

# initialise deeper model and print summary
deeper_cnn = deeper_model(input_shape=(51, 51, 1), num_classes=len(class_names))

# check deeper model architecture
deeper_cnn.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 51, 51, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 32)	0
conv2d_2 (Conv2D)	(None, 25, 25, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_3 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 128)	0
flatten_1 (Flatten)	(None, 4608)	0
dense_1 (Dense)	(None, 64)	294,976
dense_2 (Dense)	(None, 5)	325

Total params: 387,973 (1.48 MB)

Trainable params: 387,973 (1.48 MB)

Non-trainable params: 0 (0.00 B)

```
[ ]: LEARNING_RATE = 1e-4 # Some common values are 1e-3 (0.001) or 1e-5 (0.00001)
BATCH_SIZE = 32 # train with this many images per iteration [5, 10, or 20 might
    ↳ be good for a trial run]
NUM_EPOCHS = 25 # how many epochs to train for (each epoch visits the training
    ↳ data once) [5 might be good for a trial run]

def train_model(model, training_ds, validation_ds, lr, epochs):
    # model training config
    model.compile(
        optimizer=keras.optimizers.Adam(lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    training_history = model.fit(
        training_ds,
        validation_data=validation_ds,
        epochs=epochs
    )
    return training_history

# 1. train shallow CNN
# train_history will store the metrics for each epoch, for use in generating
    ↳ graphs
shallow_train = train_model(shallow_cnn, training_ds, validation_ds,
    ↳ LEARNING_RATE, NUM_EPOCHS)
```

Epoch 1/25

10/10 86s 9s/step -

accuracy: 0.2733 - loss: 8.0573 - val_accuracy: 0.2344 - val_loss: 6.6348

Epoch 2/25

10/10 2s 187ms/step -

accuracy: 0.2633 - loss: 5.0148 - val_accuracy: 0.3125 - val_loss: 4.7445

Epoch 3/25

10/10 2s 158ms/step -

accuracy: 0.3200 - loss: 3.3508 - val_accuracy: 0.3359 - val_loss: 4.0546

Epoch 4/25

10/10 2s 122ms/step -

accuracy: 0.3900 - loss: 2.7251 - val_accuracy: 0.3828 - val_loss: 3.0344

Epoch 5/25

10/10 1s 122ms/step -
accuracy: 0.3967 - loss: 2.4411 - val_accuracy: 0.3828 - val_loss: 3.1635
Epoch 6/25

10/10 1s 123ms/step -
accuracy: 0.4433 - loss: 2.1070 - val_accuracy: 0.4609 - val_loss: 2.4637
Epoch 7/25

10/10 1s 120ms/step -
accuracy: 0.4633 - loss: 1.7449 - val_accuracy: 0.3203 - val_loss: 2.8837
Epoch 8/25

10/10 1s 118ms/step -
accuracy: 0.4633 - loss: 2.0466 - val_accuracy: 0.4844 - val_loss: 3.0052
Epoch 9/25

10/10 1s 134ms/step -
accuracy: 0.4567 - loss: 2.2497 - val_accuracy: 0.4844 - val_loss: 2.1976
Epoch 10/25

10/10 1s 146ms/step -
accuracy: 0.4900 - loss: 1.7094 - val_accuracy: 0.4922 - val_loss: 2.0832
Epoch 11/25

10/10 3s 206ms/step -
accuracy: 0.5533 - loss: 1.4022 - val_accuracy: 0.4922 - val_loss: 1.8972
Epoch 12/25

10/10 2s 145ms/step -
accuracy: 0.5700 - loss: 1.2386 - val_accuracy: 0.5312 - val_loss: 2.0529
Epoch 13/25

10/10 1s 134ms/step -
accuracy: 0.6267 - loss: 1.0299 - val_accuracy: 0.5000 - val_loss: 1.8774
Epoch 14/25

10/10 1s 136ms/step -
accuracy: 0.6167 - loss: 1.0790 - val_accuracy: 0.4688 - val_loss: 2.0746
Epoch 15/25

10/10 3s 140ms/step -
accuracy: 0.5667 - loss: 1.1216 - val_accuracy: 0.5469 - val_loss: 2.0960
Epoch 16/25

10/10 1s 146ms/step -
accuracy: 0.6400 - loss: 0.9710 - val_accuracy: 0.5391 - val_loss: 1.7366
Epoch 17/25

10/10 1s 123ms/step -
accuracy: 0.6433 - loss: 0.8765 - val_accuracy: 0.6016 - val_loss: 1.7157
Epoch 18/25

10/10 2s 161ms/step -
accuracy: 0.6567 - loss: 0.8300 - val_accuracy: 0.6016 - val_loss: 1.6843
Epoch 19/25

10/10 3s 192ms/step -
accuracy: 0.6567 - loss: 0.8486 - val_accuracy: 0.4766 - val_loss: 1.9437
Epoch 20/25

10/10 2s 123ms/step -
accuracy: 0.6533 - loss: 0.9001 - val_accuracy: 0.6016 - val_loss: 1.7014
Epoch 21/25

```

10/10          1s 120ms/step -
accuracy: 0.6933 - loss: 0.7799 - val_accuracy: 0.5781 - val_loss: 1.6953
Epoch 22/25
10/10          1s 133ms/step -
accuracy: 0.7067 - loss: 0.7237 - val_accuracy: 0.5938 - val_loss: 1.6312
Epoch 23/25
10/10          1s 128ms/step -
accuracy: 0.7067 - loss: 0.7305 - val_accuracy: 0.5625 - val_loss: 1.6995
Epoch 24/25
10/10          1s 142ms/step -
accuracy: 0.7333 - loss: 0.7507 - val_accuracy: 0.5703 - val_loss: 1.6163
Epoch 25/25
10/10          2s 157ms/step -
accuracy: 0.7133 - loss: 0.7700 - val_accuracy: 0.5703 - val_loss: 1.6293

```

```

[ ]: # 2. train deeper CNN
deeper_train = train_model(deeper_cnn, training_ds, validation_ds,
↪LEARNING_RATE, NUM_EPOCHS)

```

```

Epoch 1/25
10/10          7s 411ms/step -
accuracy: 0.2067 - loss: 4.8530 - val_accuracy: 0.2109 - val_loss: 2.2232
Epoch 2/25
10/10          3s 247ms/step -
accuracy: 0.2000 - loss: 2.1757 - val_accuracy: 0.3359 - val_loss: 1.4772
Epoch 3/25
10/10          3s 266ms/step -
accuracy: 0.3267 - loss: 1.6261 - val_accuracy: 0.3203 - val_loss: 1.5591
Epoch 4/25
10/10          3s 240ms/step -
accuracy: 0.3967 - loss: 1.5165 - val_accuracy: 0.4766 - val_loss: 1.3781
Epoch 5/25
10/10          4s 351ms/step -
accuracy: 0.4367 - loss: 1.3853 - val_accuracy: 0.6094 - val_loss: 1.1998
Epoch 6/25
10/10          4s 338ms/step -
accuracy: 0.5167 - loss: 1.2365 - val_accuracy: 0.4609 - val_loss: 1.1703
Epoch 7/25
10/10          3s 247ms/step -
accuracy: 0.5533 - loss: 1.1324 - val_accuracy: 0.5547 - val_loss: 1.1181
Epoch 8/25
10/10          3s 239ms/step -
accuracy: 0.5567 - loss: 1.1228 - val_accuracy: 0.4922 - val_loss: 1.1237
Epoch 9/25
10/10          3s 246ms/step -
accuracy: 0.5567 - loss: 1.0674 - val_accuracy: 0.5234 - val_loss: 1.0725
Epoch 10/25
10/10          3s 273ms/step -

```

accuracy: 0.6100 - loss: 1.0310 - val_accuracy: 0.6094 - val_loss: 1.0264
 Epoch 11/25
 10/10 5s 226ms/step -
 accuracy: 0.6633 - loss: 0.9898 - val_accuracy: 0.5234 - val_loss: 1.0496
 Epoch 12/25
 10/10 3s 255ms/step -
 accuracy: 0.6033 - loss: 1.0065 - val_accuracy: 0.6328 - val_loss: 1.0050
 Epoch 13/25
 10/10 3s 248ms/step -
 accuracy: 0.6500 - loss: 0.9487 - val_accuracy: 0.5391 - val_loss: 1.0190
 Epoch 14/25
 10/10 3s 260ms/step -
 accuracy: 0.6567 - loss: 0.9214 - val_accuracy: 0.6406 - val_loss: 0.9229
 Epoch 15/25
 10/10 5s 227ms/step -
 accuracy: 0.6500 - loss: 0.8858 - val_accuracy: 0.5156 - val_loss: 1.0370
 Epoch 16/25
 10/10 2s 218ms/step -
 accuracy: 0.6867 - loss: 0.9358 - val_accuracy: 0.6250 - val_loss: 0.9022
 Epoch 17/25
 10/10 2s 223ms/step -
 accuracy: 0.6967 - loss: 0.8683 - val_accuracy: 0.6172 - val_loss: 0.9027
 Epoch 18/25
 10/10 3s 224ms/step -
 accuracy: 0.6800 - loss: 0.8215 - val_accuracy: 0.6641 - val_loss: 0.8900
 Epoch 19/25
 10/10 4s 416ms/step -
 accuracy: 0.6900 - loss: 0.8480 - val_accuracy: 0.6562 - val_loss: 0.8345
 Epoch 20/25
 10/10 3s 254ms/step -
 accuracy: 0.7433 - loss: 0.7943 - val_accuracy: 0.5547 - val_loss: 0.9476
 Epoch 21/25
 10/10 2s 229ms/step -
 accuracy: 0.7300 - loss: 0.7898 - val_accuracy: 0.6016 - val_loss: 0.8881
 Epoch 22/25
 10/10 3s 247ms/step -
 accuracy: 0.7433 - loss: 0.7496 - val_accuracy: 0.5859 - val_loss: 0.8854
 Epoch 23/25
 10/10 2s 222ms/step -
 accuracy: 0.6933 - loss: 0.7592 - val_accuracy: 0.6172 - val_loss: 0.8344
 Epoch 24/25
 10/10 3s 290ms/step -
 accuracy: 0.7233 - loss: 0.7379 - val_accuracy: 0.6172 - val_loss: 0.8643
 Epoch 25/25
 10/10 4s 354ms/step -
 accuracy: 0.7300 - loss: 0.7285 - val_accuracy: 0.7109 - val_loss: 0.7554


```
[ ]: test_dataset_path = Path('/content/drive/MyDrive/BMET5933/WEEK_10/
    ↪hep2img_testset')

def read_img(img_path):
    # read image and convert into np array
    img = np.array(Image.open(img_path))

    # expand channel dim
    if img.ndim == 2:
        img = img[..., np.newaxis]
    img = img / 255.0
    img = tf.image.resize(img, (51, 51))
    return img

def model_predict(model, image, class_names: list):
    image = np.array(image)

    if image.ndim == 2:
        # expand channel dim and norm
        image = np.expand_dims(image, axis = 0)
        image = image / 255.0
        image = tf.image.resize(51, 51)

    # predict image label
    prediction = model.predict(image)
    label = np.argmax(prediction, axis = 1)
    return label

def evaluate_model(model, test_ds, class_names: list):
    # get true labels and predicted labels
    true_labels = []
    pred_labels = []
    for image, label in test_ds:
        true_labels.extend(label.numpy())
        pred_label = model_predict(model, image, class_names)
        pred_labels.extend(pred_label)

    # calculate accuracy
    correct = sum(p == t for p, t in zip(pred_labels, true_labels))
    total = len(true_labels)
    accuracy = correct / total
    return accuracy
```

AI acknowledge: chatgpt5.5 used here for suggesting how to save model.

```
[ ]: # create test dataset
test_ds = keras.preprocessing.image_dataset_from_directory(
```

```

        str(test_dataset_path),
        labels='inferred',
        label_mode='int',
        batch_size=BATCH_SIZE,
        image_size=RESCALED_IMAGE_SIZE,
        shuffle=False,
        color_mode = 'grayscale'
    )

    # evaluate shallow CNN on test set
    shallow_accuracy = evaluate_model(shallow_cnn, test_ds, class_names)
    print(f"Shallow CNN Test Accuracy: {shallow_accuracy:.4f}")

    #evaluate deeper CNN on test set
    deeper_accuracy = evaluate_model(deeper_cnn, test_ds, class_names)
    print(f"Deeper CNN Test Accuracy: {deeper_accuracy:.4f}")

    # save trained models
    shallow_cnn.save('/content/drive/MyDrive/BMET5933/WEEK_10/shallow_cnn.keras')
    deeper_cnn.save('/content/drive/MyDrive/BMET5933/WEEK_10/deeper_cnn.keras')
    print("Models saved to Google Drive.")

```

Found 25 files belonging to 5 classes.

```

1/1          0s 88ms/step
Shallow CNN Test Accuracy: 0.3600
1/1          0s 131ms/step
Deeper CNN Test Accuracy: 1.0000
Models saved to Google Drive.

```

1.1 Main model comparison

Both CNN models were able to learn from the training data. For the shallow CNN, the final training accuracy was about 0.71 and the final validation accuracy was about 0.57. For the deeper CNN, the final training accuracy was about 0.73 and the final validation accuracy was about 0.71.

On the held-out test set, the shallow CNN got 0.36 accuracy and the deeper CNN got 1.00 accuracy. In my result, the deeper CNN worked much better. I think this is because the deeper CNN has more convolution layers, so it can learn more useful image features from the cell images. However, the test set only has 25 images, so the 1.00 accuracy may change if I use another test split.

2 Challenge part:

2.1 1.replace maxpool with avgpool

```

[ ]: # use avg pooling
def avgpool_shallow_model(input_shape, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),

```

```

        layers.Conv2D(16, kernel_size=(5, 5), activation='relu'),
        layers.AveragePooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(num_classes, activation='softmax'),
    ])
    return model

avg_pool_model = avgpool_shallow_model(input_shape=(51, 51, 1),
    ↪ num_classes=len(class_names))

# train avg pooling model
avg_pool_model_training = train_model(avg_pool_model, training_ds,
    ↪ validation_ds, LEARNING_RATE, NUM_EPOCHS)

# evaluate average pooling model on test set
avg_pool_accuracy = evaluate_model(avg_pool_model, test_ds, class_names)
print(f"Average Pooling CNN Test Accuracy: {avg_pool_accuracy:.4f}")

```

```

Epoch 1/25
10/10          2s 132ms/step -
accuracy: 0.1733 - loss: 8.2721 - val_accuracy: 0.1875 - val_loss: 5.2101
Epoch 2/25
10/10          1s 128ms/step -
accuracy: 0.2100 - loss: 4.4858 - val_accuracy: 0.1641 - val_loss: 4.2672
Epoch 3/25
10/10          2s 153ms/step -
accuracy: 0.2367 - loss: 3.3531 - val_accuracy: 0.2422 - val_loss: 3.8099
Epoch 4/25
10/10          2s 148ms/step -
accuracy: 0.2600 - loss: 3.2364 - val_accuracy: 0.2656 - val_loss: 3.7724
Epoch 5/25
10/10          2s 159ms/step -
accuracy: 0.2667 - loss: 2.7067 - val_accuracy: 0.3047 - val_loss: 2.9374
Epoch 6/25
10/10          1s 151ms/step -
accuracy: 0.3367 - loss: 2.2197 - val_accuracy: 0.3125 - val_loss: 2.9502
Epoch 7/25
10/10          1s 108ms/step -
accuracy: 0.3667 - loss: 2.0778 - val_accuracy: 0.3047 - val_loss: 2.7297
Epoch 8/25
10/10          1s 118ms/step -
accuracy: 0.3867 - loss: 2.0774 - val_accuracy: 0.3672 - val_loss: 2.7081
Epoch 9/25
10/10          1s 117ms/step -
accuracy: 0.4267 - loss: 1.7026 - val_accuracy: 0.4297 - val_loss: 2.2764
Epoch 10/25
10/10          1s 121ms/step -

```

```

accuracy: 0.4833 - loss: 1.6544 - val_accuracy: 0.4141 - val_loss: 2.0568
Epoch 11/25
10/10          1s 122ms/step -
accuracy: 0.5033 - loss: 1.4821 - val_accuracy: 0.4297 - val_loss: 1.8265
Epoch 12/25
10/10          1s 113ms/step -
accuracy: 0.5667 - loss: 1.2428 - val_accuracy: 0.4453 - val_loss: 1.7929
Epoch 13/25
10/10          1s 110ms/step -
accuracy: 0.5667 - loss: 1.1361 - val_accuracy: 0.5234 - val_loss: 1.6259
Epoch 14/25
10/10          2s 161ms/step -
accuracy: 0.5700 - loss: 1.1254 - val_accuracy: 0.4766 - val_loss: 1.6969
Epoch 15/25
10/10          3s 182ms/step -
accuracy: 0.5700 - loss: 1.0740 - val_accuracy: 0.5234 - val_loss: 1.6367
Epoch 16/25
10/10          1s 109ms/step -
accuracy: 0.6533 - loss: 0.9921 - val_accuracy: 0.4531 - val_loss: 1.8557
Epoch 17/25
10/10          1s 108ms/step -
accuracy: 0.6133 - loss: 1.0871 - val_accuracy: 0.5469 - val_loss: 1.5027
Epoch 18/25
10/10          1s 111ms/step -
accuracy: 0.6800 - loss: 0.8138 - val_accuracy: 0.5938 - val_loss: 1.4068
Epoch 19/25
10/10          1s 113ms/step -
accuracy: 0.6700 - loss: 0.8731 - val_accuracy: 0.5000 - val_loss: 1.5110
Epoch 20/25
10/10          1s 116ms/step -
accuracy: 0.7167 - loss: 0.7542 - val_accuracy: 0.5234 - val_loss: 1.4610
Epoch 21/25
10/10          1s 115ms/step -
accuracy: 0.7300 - loss: 0.7049 - val_accuracy: 0.6172 - val_loss: 1.3093
Epoch 22/25
10/10          1s 110ms/step -
accuracy: 0.7667 - loss: 0.5858 - val_accuracy: 0.5391 - val_loss: 1.5027
Epoch 23/25
10/10          1s 113ms/step -
accuracy: 0.7733 - loss: 0.5893 - val_accuracy: 0.5703 - val_loss: 1.3510
Epoch 24/25
10/10          1s 141ms/step -
accuracy: 0.7900 - loss: 0.5592 - val_accuracy: 0.5703 - val_loss: 1.3855
Epoch 25/25
10/10          2s 163ms/step -
accuracy: 0.7500 - loss: 0.5752 - val_accuracy: 0.5625 - val_loss: 1.4761
1/1            0s 116ms/step
Average Pooling CNN Test Accuracy: 0.5600

```

2.1.1 Average pooling conclusion

The average pooling model got 0.56 test accuracy, while the original shallow CNN got 0.36. In this run, average pooling worked better than the original max pooling shallow model. I think average pooling may keep more general brightness and texture information from the cell image regions. However, it was still lower than the deeper CNN result, so it only improved the shallow model in this experiment.

2.2 2.Parrameter changing

AI acknowledge: chatgpt5.5 used here for giving suggestions on encapsulating function of 'train_model' and polishing explanation.

2.3 Parameter changing note

For this part, I used the original shallow CNN as the baseline. I only changed one convolution parameter each time, so the results are easier to compare: 1. kernel size: 5 -> 7 2. number of filters: 16 -> 32 3. stride: 1 -> 2 4. padding: same -> valid

```
[ ]: # model variations with config
# filter size 7
shallow_model_filter7 = shallow_model(input_shape=(51, 51, 1),
    ↪num_classes=len(class_names), filter_size=7)
filter7_train = train_model(shallow_model_filter7, training_ds, validation_ds,
    ↪LEARNING_RATE, NUM_EPOCHS)
```

Epoch 1/25

10/10 3s 146ms/step -

accuracy: 0.1900 - loss: 7.6549 - val_accuracy: 0.2422 - val_loss: 5.6149

Epoch 2/25

10/10 2s 162ms/step -

accuracy: 0.2067 - loss: 4.8882 - val_accuracy: 0.2500 - val_loss: 4.3636

Epoch 3/25

10/10 2s 175ms/step -

accuracy: 0.2900 - loss: 3.4926 - val_accuracy: 0.2891 - val_loss: 3.0414

Epoch 4/25

10/10 1s 136ms/step -

accuracy: 0.3300 - loss: 2.6809 - val_accuracy: 0.3203 - val_loss: 2.6439

Epoch 5/25

10/10 2s 176ms/step -

accuracy: 0.3900 - loss: 2.2548 - val_accuracy: 0.4062 - val_loss: 2.6756

Epoch 6/25

10/10 1s 132ms/step -

accuracy: 0.4333 - loss: 2.0672 - val_accuracy: 0.4062 - val_loss: 2.3698

Epoch 7/25

10/10 2s 206ms/step -

accuracy: 0.4533 - loss: 1.8005 - val_accuracy: 0.4062 - val_loss: 2.2992

Epoch 8/25

10/10 3s 227ms/step -

accuracy: 0.5100 - loss: 1.6234 - val_accuracy: 0.5078 - val_loss: 2.0162
 Epoch 9/25
 10/10 2s 144ms/step -
 accuracy: 0.5067 - loss: 1.5149 - val_accuracy: 0.4844 - val_loss: 1.9319
 Epoch 10/25
 10/10 1s 131ms/step -
 accuracy: 0.5400 - loss: 1.3366 - val_accuracy: 0.4844 - val_loss: 1.8191
 Epoch 11/25
 10/10 2s 145ms/step -
 accuracy: 0.5667 - loss: 1.4244 - val_accuracy: 0.5156 - val_loss: 1.6912
 Epoch 12/25
 10/10 2s 137ms/step -
 accuracy: 0.5500 - loss: 1.2560 - val_accuracy: 0.5000 - val_loss: 1.7173
 Epoch 13/25
 10/10 1s 141ms/step -
 accuracy: 0.5800 - loss: 1.0605 - val_accuracy: 0.5234 - val_loss: 1.5963
 Epoch 14/25
 10/10 1s 128ms/step -
 accuracy: 0.5500 - loss: 1.2192 - val_accuracy: 0.5078 - val_loss: 1.7217
 Epoch 15/25
 10/10 4s 237ms/step -
 accuracy: 0.5500 - loss: 1.1926 - val_accuracy: 0.5234 - val_loss: 1.4344
 Epoch 16/25
 10/10 2s 141ms/step -
 accuracy: 0.6133 - loss: 1.1888 - val_accuracy: 0.5469 - val_loss: 1.5243
 Epoch 17/25
 10/10 1s 138ms/step -
 accuracy: 0.6367 - loss: 0.9191 - val_accuracy: 0.5234 - val_loss: 1.4523
 Epoch 18/25
 10/10 1s 138ms/step -
 accuracy: 0.6233 - loss: 0.9090 - val_accuracy: 0.5469 - val_loss: 1.3026
 Epoch 19/25
 10/10 1s 145ms/step -
 accuracy: 0.6767 - loss: 0.8230 - val_accuracy: 0.5391 - val_loss: 1.5586
 Epoch 20/25
 10/10 2s 136ms/step -
 accuracy: 0.6700 - loss: 0.8654 - val_accuracy: 0.5547 - val_loss: 1.3378
 Epoch 21/25
 10/10 1s 138ms/step -
 accuracy: 0.6500 - loss: 0.8281 - val_accuracy: 0.6328 - val_loss: 1.3304
 Epoch 22/25
 10/10 2s 199ms/step -
 accuracy: 0.7200 - loss: 0.7245 - val_accuracy: 0.5859 - val_loss: 1.1947
 Epoch 23/25
 10/10 3s 213ms/step -
 accuracy: 0.7067 - loss: 0.7060 - val_accuracy: 0.5156 - val_loss: 1.3376
 Epoch 24/25
 10/10 2s 153ms/step -

accuracy: 0.6800 - loss: 0.8743 - val_accuracy: 0.5469 - val_loss: 1.5558
Epoch 25/25
10/10 1s 139ms/step -
accuracy: 0.7000 - loss: 0.7233 - val_accuracy: 0.5781 - val_loss: 1.3642

```
[ ]: # stride 2
shallow_model_stride2 = shallow_model(input_shape=(51, 51, 1),
    ↳ num_classes=len(class_names), strides=2)
stride2_train = train_model(shallow_model_stride2, training_ds, validation_ds,
    ↳ LEARNING_RATE, NUM_EPOCHS)
```

Epoch 1/25
10/10 2s 126ms/step -
accuracy: 0.1633 - loss: 17.8903 - val_accuracy: 0.2109 - val_loss: 10.1448
Epoch 2/25
10/10 1s 102ms/step -
accuracy: 0.1533 - loss: 8.7861 - val_accuracy: 0.0938 - val_loss: 8.7723
Epoch 3/25
10/10 1s 101ms/step -
accuracy: 0.1567 - loss: 7.1362 - val_accuracy: 0.2422 - val_loss: 7.4635
Epoch 4/25
10/10 1s 101ms/step -
accuracy: 0.1400 - loss: 6.3389 - val_accuracy: 0.1250 - val_loss: 7.3332
Epoch 5/25
10/10 1s 110ms/step -
accuracy: 0.1700 - loss: 5.8349 - val_accuracy: 0.2188 - val_loss: 6.5884
Epoch 6/25
10/10 1s 126ms/step -
accuracy: 0.1733 - loss: 5.3653 - val_accuracy: 0.1797 - val_loss: 6.2389
Epoch 7/25
10/10 1s 128ms/step -
accuracy: 0.1500 - loss: 5.1810 - val_accuracy: 0.2031 - val_loss: 5.8251
Epoch 8/25
10/10 2s 106ms/step -
accuracy: 0.2200 - loss: 4.7210 - val_accuracy: 0.2031 - val_loss: 5.5974
Epoch 9/25
10/10 1s 103ms/step -
accuracy: 0.1933 - loss: 4.4359 - val_accuracy: 0.2109 - val_loss: 5.2888
Epoch 10/25
10/10 1s 110ms/step -
accuracy: 0.2567 - loss: 4.2388 - val_accuracy: 0.2422 - val_loss: 5.0688
Epoch 11/25
10/10 1s 100ms/step -
accuracy: 0.2667 - loss: 4.0295 - val_accuracy: 0.2891 - val_loss: 4.7364
Epoch 12/25
10/10 1s 99ms/step -
accuracy: 0.2367 - loss: 3.7799 - val_accuracy: 0.2500 - val_loss: 4.6200
Epoch 13/25

```

10/10          1s 98ms/step -
accuracy: 0.2867 - loss: 3.8831 - val_accuracy: 0.2891 - val_loss: 4.5379
Epoch 14/25
10/10          1s 94ms/step -
accuracy: 0.2733 - loss: 3.4541 - val_accuracy: 0.2500 - val_loss: 4.5408
Epoch 15/25
10/10          1s 101ms/step -
accuracy: 0.2867 - loss: 3.5912 - val_accuracy: 0.2891 - val_loss: 4.6026
Epoch 16/25
10/10          1s 121ms/step -
accuracy: 0.3067 - loss: 3.2002 - val_accuracy: 0.2422 - val_loss: 4.2919
Epoch 17/25
10/10          1s 128ms/step -
accuracy: 0.3067 - loss: 3.3234 - val_accuracy: 0.3203 - val_loss: 3.9308
Epoch 18/25
10/10          1s 131ms/step -
accuracy: 0.3167 - loss: 2.9406 - val_accuracy: 0.3281 - val_loss: 3.8928
Epoch 19/25
10/10          1s 140ms/step -
accuracy: 0.3567 - loss: 2.9920 - val_accuracy: 0.2344 - val_loss: 4.0260
Epoch 20/25
10/10          1s 110ms/step -
accuracy: 0.3333 - loss: 2.7790 - val_accuracy: 0.3281 - val_loss: 3.6005
Epoch 21/25
10/10          1s 101ms/step -
accuracy: 0.3700 - loss: 2.6041 - val_accuracy: 0.2734 - val_loss: 3.7967
Epoch 22/25
10/10          1s 110ms/step -
accuracy: 0.3367 - loss: 2.6365 - val_accuracy: 0.3359 - val_loss: 3.4057
Epoch 23/25
10/10          1s 102ms/step -
accuracy: 0.3467 - loss: 2.4924 - val_accuracy: 0.3516 - val_loss: 3.4841
Epoch 24/25
10/10          1s 103ms/step -
accuracy: 0.4133 - loss: 2.2908 - val_accuracy: 0.3516 - val_loss: 3.3646
Epoch 25/25
10/10          1s 102ms/step -
accuracy: 0.4000 - loss: 2.2483 - val_accuracy: 0.3672 - val_loss: 3.1204

```

```

[ ]: # valid padding
shallow_model_padding_valid = shallow_model(input_shape=(51, 51, 1),
    ↳ num_classes=len(class_names), padding='valid')
padding_valid_train = train_model(shallow_model_padding_valid, training_ds,
    ↳ validation_ds, LEARNING_RATE, NUM_EPOCHS)

```

```

Epoch 1/25
10/10          2s 140ms/step -
accuracy: 0.1367 - loss: 11.1420 - val_accuracy: 0.1641 - val_loss: 9.4721

```


Epoch 2/25
10/10 2s 161ms/step -
accuracy: 0.1667 - loss: 5.4833 - val_accuracy: 0.1953 - val_loss: 4.5969
Epoch 3/25
10/10 2s 158ms/step -
accuracy: 0.1767 - loss: 3.8064 - val_accuracy: 0.1328 - val_loss: 4.4025
Epoch 4/25
10/10 3s 173ms/step -
accuracy: 0.2133 - loss: 3.1757 - val_accuracy: 0.2500 - val_loss: 3.5995
Epoch 5/25
10/10 1s 120ms/step -
accuracy: 0.2033 - loss: 2.8825 - val_accuracy: 0.2188 - val_loss: 3.7289
Epoch 6/25
10/10 1s 117ms/step -
accuracy: 0.2767 - loss: 2.4703 - val_accuracy: 0.1719 - val_loss: 3.5164
Epoch 7/25
10/10 1s 125ms/step -
accuracy: 0.3033 - loss: 2.1980 - val_accuracy: 0.2656 - val_loss: 3.1941
Epoch 8/25
10/10 1s 124ms/step -
accuracy: 0.2933 - loss: 2.2233 - val_accuracy: 0.2891 - val_loss: 3.0108
Epoch 9/25
10/10 1s 113ms/step -
accuracy: 0.3667 - loss: 2.0559 - val_accuracy: 0.3203 - val_loss: 2.7763
Epoch 10/25
10/10 2s 156ms/step -
accuracy: 0.3633 - loss: 1.8129 - val_accuracy: 0.4062 - val_loss: 2.6938
Epoch 11/25
10/10 1s 118ms/step -
accuracy: 0.4400 - loss: 1.6350 - val_accuracy: 0.3672 - val_loss: 2.5508
Epoch 12/25
10/10 1s 112ms/step -
accuracy: 0.4400 - loss: 1.6035 - val_accuracy: 0.3906 - val_loss: 2.4618
Epoch 13/25
10/10 2s 199ms/step -
accuracy: 0.5200 - loss: 1.4412 - val_accuracy: 0.4062 - val_loss: 2.3486
Epoch 14/25
10/10 2s 176ms/step -
accuracy: 0.4633 - loss: 1.4287 - val_accuracy: 0.4375 - val_loss: 2.8470
Epoch 15/25
10/10 1s 134ms/step -
accuracy: 0.4900 - loss: 1.6759 - val_accuracy: 0.4531 - val_loss: 2.7008
Epoch 16/25
10/10 1s 115ms/step -
accuracy: 0.4567 - loss: 1.4863 - val_accuracy: 0.4453 - val_loss: 2.7471
Epoch 17/25
10/10 1s 119ms/step -
accuracy: 0.4800 - loss: 1.4582 - val_accuracy: 0.4297 - val_loss: 2.6564

```

Epoch 18/25
10/10          1s 115ms/step -
accuracy: 0.5500 - loss: 1.2350 - val_accuracy: 0.4844 - val_loss: 2.0837
Epoch 19/25
10/10          1s 115ms/step -
accuracy: 0.5967 - loss: 1.1411 - val_accuracy: 0.4141 - val_loss: 2.3010
Epoch 20/25
10/10          1s 121ms/step -
accuracy: 0.5733 - loss: 1.0019 - val_accuracy: 0.5000 - val_loss: 2.1448
Epoch 21/25
10/10          2s 161ms/step -
accuracy: 0.6167 - loss: 0.9820 - val_accuracy: 0.4766 - val_loss: 2.2594
Epoch 22/25
10/10          1s 116ms/step -
accuracy: 0.5733 - loss: 1.0793 - val_accuracy: 0.5078 - val_loss: 2.5088
Epoch 23/25
10/10          1s 141ms/step -
accuracy: 0.6167 - loss: 1.0679 - val_accuracy: 0.5078 - val_loss: 2.2395
Epoch 24/25
10/10          2s 173ms/step -
accuracy: 0.6567 - loss: 0.8917 - val_accuracy: 0.5078 - val_loss: 2.0361
Epoch 25/25
10/10          2s 168ms/step -
accuracy: 0.6667 - loss: 0.8723 - val_accuracy: 0.5312 - val_loss: 1.9108

```

```

[ ]: # number of filters 32
shallow_model_filters32 = shallow_model(input_shape=(51, 51, 1),
    ↪num_classes=len(class_names), num_filters=32)
filters32_train = train_model(shallow_model_filters32, training_ds,
    ↪validation_ds, LEARNING_RATE, NUM_EPOCHS)

```

```

Epoch 1/25
10/10          3s 144ms/step -
accuracy: 0.2000 - loss: 12.8080 - val_accuracy: 0.2422 - val_loss: 6.4159
Epoch 2/25
10/10          3s 149ms/step -
accuracy: 0.2467 - loss: 5.6179 - val_accuracy: 0.1797 - val_loss: 5.7305
Epoch 3/25
10/10          1s 132ms/step -
accuracy: 0.3067 - loss: 4.1700 - val_accuracy: 0.1797 - val_loss: 4.0858
Epoch 4/25
10/10          1s 131ms/step -
accuracy: 0.3000 - loss: 2.7848 - val_accuracy: 0.3047 - val_loss: 2.7479
Epoch 5/25
10/10          1s 130ms/step -
accuracy: 0.4133 - loss: 2.3124 - val_accuracy: 0.3125 - val_loss: 2.9756
Epoch 6/25
10/10          2s 180ms/step -

```

accuracy: 0.3633 - loss: 2.4137 - val_accuracy: 0.3359 - val_loss: 3.2520
 Epoch 7/25
 10/10 2s 213ms/step -
 accuracy: 0.3800 - loss: 2.6231 - val_accuracy: 0.3438 - val_loss: 2.4690
 Epoch 8/25
 10/10 2s 187ms/step -
 accuracy: 0.4733 - loss: 1.9972 - val_accuracy: 0.3984 - val_loss: 2.4823
 Epoch 9/25
 10/10 1s 131ms/step -
 accuracy: 0.4867 - loss: 1.7550 - val_accuracy: 0.5156 - val_loss: 1.7051
 Epoch 10/25
 10/10 1s 125ms/step -
 accuracy: 0.4833 - loss: 1.8363 - val_accuracy: 0.4531 - val_loss: 3.3004
 Epoch 11/25
 10/10 1s 130ms/step -
 accuracy: 0.5133 - loss: 2.1878 - val_accuracy: 0.4141 - val_loss: 2.5761
 Epoch 12/25
 10/10 1s 131ms/step -
 accuracy: 0.4933 - loss: 1.5742 - val_accuracy: 0.4609 - val_loss: 1.7951
 Epoch 13/25
 10/10 1s 130ms/step -
 accuracy: 0.5267 - loss: 1.3934 - val_accuracy: 0.3984 - val_loss: 1.8228
 Epoch 14/25
 10/10 1s 134ms/step -
 accuracy: 0.5700 - loss: 1.1272 - val_accuracy: 0.4219 - val_loss: 1.5841
 Epoch 15/25
 10/10 1s 129ms/step -
 accuracy: 0.6467 - loss: 1.0409 - val_accuracy: 0.5078 - val_loss: 1.4114
 Epoch 16/25
 10/10 2s 228ms/step -
 accuracy: 0.6700 - loss: 0.9204 - val_accuracy: 0.5312 - val_loss: 1.4797
 Epoch 17/25
 10/10 2s 213ms/step -
 accuracy: 0.6367 - loss: 1.0038 - val_accuracy: 0.5625 - val_loss: 1.3347
 Epoch 18/25
 10/10 2s 132ms/step -
 accuracy: 0.6600 - loss: 0.8793 - val_accuracy: 0.6016 - val_loss: 1.3143
 Epoch 19/25
 10/10 1s 132ms/step -
 accuracy: 0.6433 - loss: 0.9093 - val_accuracy: 0.5938 - val_loss: 1.3362
 Epoch 20/25
 10/10 1s 115ms/step -
 accuracy: 0.6633 - loss: 0.8764 - val_accuracy: 0.5625 - val_loss: 1.2681
 Epoch 21/25
 10/10 1s 128ms/step -
 accuracy: 0.6900 - loss: 0.8115 - val_accuracy: 0.4922 - val_loss: 1.8491
 Epoch 22/25
 10/10 1s 132ms/step -

```

accuracy: 0.6567 - loss: 0.9711 - val_accuracy: 0.5391 - val_loss: 1.3469
Epoch 23/25
10/10          1s 133ms/step -
accuracy: 0.6900 - loss: 0.8583 - val_accuracy: 0.5859 - val_loss: 1.0961
Epoch 24/25
10/10          1s 135ms/step -
accuracy: 0.7000 - loss: 0.7845 - val_accuracy: 0.5781 - val_loss: 1.2208
Epoch 25/25
10/10          2s 173ms/step -
accuracy: 0.7033 - loss: 0.7425 - val_accuracy: 0.5703 - val_loss: 1.1503

```

```

[ ]: # evaluate model variations on test set
filter7_accuracy = evaluate_model(shallow_model_filter7, test_ds, class_names)
print(f"Shallow CNN with filter size 7 Test Accuracy: {filter7_accuracy:.4f}")

filters32_accuracy = evaluate_model(shallow_model_filters32, test_ds,
    ↪class_names)
print(f"Shallow CNN with 32 filters Test Accuracy: {filters32_accuracy:.4f}")

stride2_accuracy = evaluate_model(shallow_model_stride2, test_ds, class_names)
print(f"Shallow CNN with stride 2 Test Accuracy: {stride2_accuracy:.4f}")

padding_valid_accuracy = evaluate_model(shallow_model_padding_valid, test_ds,
    ↪class_names)
print(f"Shallow CNN with valid padding Test Accuracy: {padding_valid_accuracy:.
    ↪4f}")

challenge2_results = pd.DataFrame([
    {"change": "baseline", "filter_size": 5, "num_filters": 16, "stride":
    ↪1, "padding": "same", "test_accuracy": shallow_accuracy},
    {"change": "filter size 7", "filter_size": 7, "num_filters": 16,
    ↪"stride": 1, "padding": "same", "test_accuracy": filter7_accuracy},
    {"change": "32 filters", "filter_size": 5, "num_filters": 32, "stride":
    ↪1, "padding": "same", "test_accuracy": filters32_accuracy},
    {"change": "stride 2", "filter_size": 5, "num_filters": 16, "stride":
    ↪2, "padding": "same", "test_accuracy": stride2_accuracy},
    {"change": "valid padding", "filter_size": 5, "num_filters": 16,
    ↪"stride": 1, "padding": "valid", "test_accuracy": padding_valid_accuracy},
])

challenge2_results["accuracy_change"] = challenge2_results["test_accuracy"] -
    ↪shallow_accuracy
challenge2_results

```

```

1/1          0s 126ms/step
Shallow CNN with filter size 7 Test Accuracy: 0.6400
WARNING:tensorflow:5 out of the last 5 calls to <function

```

TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7b98b74fa2a0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```
1/1          0s 136ms/step
Shallow CNN with 32 filters Test Accuracy: 0.5600
```

WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7b98b74fab60> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

```
1/1          0s 119ms/step
Shallow CNN with stride 2 Test Accuracy: 0.3200
1/1          0s 119ms/step
Shallow CNN with valid padding Test Accuracy: 0.4400
```

[]:	change	filter_size	num_filters	stride	padding	test_accuracy \
0	baseline	5	16	1	same	0.36
1	filter size 7	7	16	1	same	0.64
2	32 filters	5	32	1	same	0.56
3	stride 2	5	16	2	same	0.32
4	valid padding	5	16	1	valid	0.44
	accuracy_change					
0	0.00					
1	0.28					
2	0.20					
3	-0.04					
4	0.08					

2.3.1 Challenge 2 conclusion

From the result table, the 7x7 filter gave the best result with 0.64 test accuracy. The 32-filter model also improved the result to 0.56, and valid padding gave 0.44. In my result, using a larger filter or more filters helped the shallow CNN learn better features. The stride 2 model got 0.32, which was the worst result. I think this is because stride 2 makes the feature map smaller too early, so the

model may miss some small cell texture details.

2.4 Challenge 3: Changing training hyperparameters

AI Acknowledge: chatgpt5.5 used in this part for polishing explanation.

```
[ ]: # challenge 3 - change learning rate
# create a new model so the comparison starts from fresh weights
shallow_model_lr_1e3 = shallow_model(input_shape=(51, 51, 1),
    ↪ num_classes=len(class_names))

# train model with learning rate 1e-3 and original epochs
lr_variation_training = train_model(shallow_model_lr_1e3, training_ds,
    ↪ validation_ds, lr=1e-3, epochs=NUM_EPOCHS)

# evaluate model with learning rate 1e-3
lr_variation_accuracy = evaluate_model(shallow_model_lr_1e3, test_ds,
    ↪ class_names)
print(f"Shallow CNN Test Accuracy with learning rate 1e-3:
    ↪ {lr_variation_accuracy:.4f}")
```

```
Epoch 1/25
10/10          3s 143ms/step -
accuracy: 0.1800 - loss: 51.0038 - val_accuracy: 0.1484 - val_loss: 14.6153
Epoch 2/25
10/10          2s 160ms/step -
accuracy: 0.2267 - loss: 12.2241 - val_accuracy: 0.3594 - val_loss: 3.7370
Epoch 3/25
10/10          2s 158ms/step -
accuracy: 0.4200 - loss: 3.6775 - val_accuracy: 0.4531 - val_loss: 2.2917
Epoch 4/25
10/10          1s 128ms/step -
accuracy: 0.4333 - loss: 1.5205 - val_accuracy: 0.3828 - val_loss: 1.8997
Epoch 5/25
10/10          1s 136ms/step -
accuracy: 0.3567 - loss: 1.4601 - val_accuracy: 0.4766 - val_loss: 1.3404
Epoch 6/25
10/10          2s 165ms/step -
accuracy: 0.5300 - loss: 1.2612 - val_accuracy: 0.4844 - val_loss: 1.3425
Epoch 7/25
10/10          2s 161ms/step -
accuracy: 0.5833 - loss: 1.0741 - val_accuracy: 0.5547 - val_loss: 1.1795
Epoch 8/25
10/10          2s 219ms/step -
accuracy: 0.6667 - loss: 0.9286 - val_accuracy: 0.5703 - val_loss: 1.1504
Epoch 9/25
10/10          2s 154ms/step -
accuracy: 0.6900 - loss: 0.7923 - val_accuracy: 0.6172 - val_loss: 1.1609
```

Epoch 10/25
10/10 1s 118ms/step -
accuracy: 0.7300 - loss: 0.7241 - val_accuracy: 0.5625 - val_loss: 1.1266
Epoch 11/25
10/10 2s 159ms/step -
accuracy: 0.6867 - loss: 0.7513 - val_accuracy: 0.5703 - val_loss: 1.2934
Epoch 12/25
10/10 2s 160ms/step -
accuracy: 0.7700 - loss: 0.6183 - val_accuracy: 0.5547 - val_loss: 1.3236
Epoch 13/25
10/10 1s 125ms/step -
accuracy: 0.8067 - loss: 0.5641 - val_accuracy: 0.5625 - val_loss: 1.2132
Epoch 14/25
10/10 2s 163ms/step -
accuracy: 0.7400 - loss: 0.6284 - val_accuracy: 0.5547 - val_loss: 1.6840
Epoch 15/25
10/10 1s 132ms/step -
accuracy: 0.7900 - loss: 0.5770 - val_accuracy: 0.5859 - val_loss: 1.2138
Epoch 16/25
10/10 1s 138ms/step -
accuracy: 0.8033 - loss: 0.5381 - val_accuracy: 0.6172 - val_loss: 1.2305
Epoch 17/25
10/10 2s 180ms/step -
accuracy: 0.8233 - loss: 0.4789 - val_accuracy: 0.5859 - val_loss: 1.1542
Epoch 18/25
10/10 2s 159ms/step -
accuracy: 0.8667 - loss: 0.4298 - val_accuracy: 0.5859 - val_loss: 1.2408
Epoch 19/25
10/10 2s 160ms/step -
accuracy: 0.8433 - loss: 0.4291 - val_accuracy: 0.6172 - val_loss: 1.2473
Epoch 20/25
10/10 1s 120ms/step -
accuracy: 0.8600 - loss: 0.4173 - val_accuracy: 0.5312 - val_loss: 1.6407
Epoch 21/25
10/10 1s 128ms/step -
accuracy: 0.8100 - loss: 0.4755 - val_accuracy: 0.5703 - val_loss: 1.4427
Epoch 22/25
10/10 1s 119ms/step -
accuracy: 0.8667 - loss: 0.3953 - val_accuracy: 0.5234 - val_loss: 1.5557
Epoch 23/25
10/10 1s 124ms/step -
accuracy: 0.8567 - loss: 0.4213 - val_accuracy: 0.5312 - val_loss: 1.4929
Epoch 24/25
10/10 1s 119ms/step -
accuracy: 0.8933 - loss: 0.3625 - val_accuracy: 0.6250 - val_loss: 1.2933
Epoch 25/25
10/10 1s 150ms/step -
accuracy: 0.8900 - loss: 0.3259 - val_accuracy: 0.6250 - val_loss: 1.2606

1/1 0s 123ms/step
Shallow CNN Test Accuracy with learning rate 1e-3: 0.6800

```
[ ]: # challenge 3 - change number of epochs
# create another new model so it does not continue from the previous model
shallow_model_epoch15 = shallow_model(input_shape=(51, 51, 1),
    ↪ num_classes=len(class_names))

# train model with original learning rate and 15 epochs
epoch_variation_training = train_model(shallow_model_epoch15, training_ds,
    ↪ validation_ds, lr=LEARNING_RATE, epochs=15)

# evaluate model with 15 epochs
epoch_variation_accuracy = evaluate_model(shallow_model_epoch15, test_ds,
    ↪ class_names)
print(f"Shallow CNN Test Accuracy with 15 epochs: {epoch_variation_accuracy:.
    ↪ 4f}")

challenge3_results = pd.DataFrame([
    {"change": "baseline", "learning_rate": LEARNING_RATE, "epochs":
    ↪ NUM_EPOCHS, "test_accuracy": shallow_accuracy},
    {"change": "learning rate 1e-3", "learning_rate": 1e-3, "epochs":
    ↪ NUM_EPOCHS, "test_accuracy": lr_variation_accuracy},
    {"change": "15 epochs", "learning_rate": LEARNING_RATE, "epochs": 15,
    ↪ "test_accuracy": epoch_variation_accuracy},
])

challenge3_results["accuracy_change"] = challenge3_results["test_accuracy"] -
    ↪ shallow_accuracy
challenge3_results
```

Epoch 1/15
10/10 3s 204ms/step -
accuracy: 0.1967 - loss: 15.9220 - val_accuracy: 0.1328 - val_loss: 9.2478
Epoch 2/15
10/10 1s 124ms/step -
accuracy: 0.1733 - loss: 7.8066 - val_accuracy: 0.1094 - val_loss: 6.0183
Epoch 3/15
10/10 1s 126ms/step -
accuracy: 0.2233 - loss: 5.2253 - val_accuracy: 0.2266 - val_loss: 4.8478
Epoch 4/15
10/10 1s 122ms/step -
accuracy: 0.2467 - loss: 4.2444 - val_accuracy: 0.2266 - val_loss: 4.3737
Epoch 5/15
10/10 1s 131ms/step -
accuracy: 0.2733 - loss: 4.0751 - val_accuracy: 0.2500 - val_loss: 3.7768
Epoch 6/15


```

10/10          2s 116ms/step -
accuracy: 0.3300 - loss: 3.1284 - val_accuracy: 0.2500 - val_loss: 3.4777
Epoch 7/15
10/10          1s 120ms/step -
accuracy: 0.3500 - loss: 2.7743 - val_accuracy: 0.2578 - val_loss: 2.8673
Epoch 8/15
10/10          2s 175ms/step -
accuracy: 0.3933 - loss: 2.3039 - val_accuracy: 0.3203 - val_loss: 2.5126
Epoch 9/15
10/10          2s 198ms/step -
accuracy: 0.4300 - loss: 2.0997 - val_accuracy: 0.3594 - val_loss: 2.4015
Epoch 10/15
10/10          2s 171ms/step -
accuracy: 0.4200 - loss: 2.1809 - val_accuracy: 0.3906 - val_loss: 1.9544
Epoch 11/15
10/10          2s 128ms/step -
accuracy: 0.4867 - loss: 1.6939 - val_accuracy: 0.3672 - val_loss: 2.4187
Epoch 12/15
10/10          1s 119ms/step -
accuracy: 0.4333 - loss: 2.1389 - val_accuracy: 0.3203 - val_loss: 2.5591
Epoch 13/15
10/10          1s 129ms/step -
accuracy: 0.4800 - loss: 1.5020 - val_accuracy: 0.4531 - val_loss: 1.6265
Epoch 14/15
10/10          2s 116ms/step -
accuracy: 0.5600 - loss: 1.2952 - val_accuracy: 0.4062 - val_loss: 1.7818
Epoch 15/15
10/10          1s 125ms/step -
accuracy: 0.5600 - loss: 1.2589 - val_accuracy: 0.4766 - val_loss: 1.4901
1/1           0s 82ms/step
Shallow CNN Test Accuracy with 15 epochs: 0.5200

```

```

[ ]:          change  learning_rate  epochs  test_accuracy  accuracy_change
0          baseline      0.0001      25          0.36          0.00
1  learning rate 1e-3      0.0010      25          0.68          0.32
2          15 epochs      0.0001      15          0.52          0.16

```

2.4.1 Challenge 3 conclusion

For the training hyperparameters, learning rate 1e-3 gave the best result in this comparison. It reached 0.68 test accuracy, which was higher than the baseline 0.36. The model trained for 15 epochs also reached 0.52, so it was also better than the baseline in this run. I think the original learning rate 1e-4 may be too slow for the shallow CNN. The results can also change because the test set is small, but in my current result the larger learning rate worked best.

2.5 Challenge 4: Deeper model

```
[ ]: def even_deeper_model(input_shape, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(5, 5), activation='relu',
        padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(5, 5), activation='relu',
        padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(128, kernel_size=(3, 3), activation='relu',
        padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(256, kernel_size=(3, 3), activation='relu',
        padding='same'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model

even_deeper_cnn = even_deeper_model(input_shape=(51, 51, 1),
    num_classes=len(class_names))
even_deeper_cnn.summary()
```

Model: "sequential_9"

Layer (type)	Output Shape	Param #
conv2d_11 (Conv2D)	(None, 51, 51, 32)	832
max_pooling2d_10 (MaxPooling2D)	(None, 25, 25, 32)	0
conv2d_12 (Conv2D)	(None, 25, 25, 64)	51,264
max_pooling2d_11 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_13 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_12 (MaxPooling2D)	(None, 6, 6, 128)	0
conv2d_14 (Conv2D)	(None, 6, 6, 256)	295,168

max_pooling2d_13 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_9 (Flatten)	(None, 2304)	0
dense_10 (Dense)	(None, 128)	295,040
dense_11 (Dense)	(None, 64)	8,256
dense_12 (Dense)	(None, 5)	325

Total params: 724,741 (2.76 MB)

Trainable params: 724,741 (2.76 MB)

Non-trainable params: 0 (0.00 B)

```
[ ]: even_deeper_train = train_model(even_deeper_cnn, training_ds, validation_ds,
    ↪LEARNING_RATE, NUM_EPOCHS)
even_deeper_accuracy = evaluate_model(even_deeper_cnn, test_ds, class_names)
print(f"Even deeper CNN Test Accuracy: {even_deeper_accuracy:.4f}")
```

Epoch 1/25

10/10 8s 408ms/step -
accuracy: 0.1967 - loss: 2.8795 - val_accuracy: 0.1953 - val_loss: 1.6555

Epoch 2/25

10/10 4s 376ms/step -
accuracy: 0.2900 - loss: 1.5073 - val_accuracy: 0.2891 - val_loss: 1.4853

Epoch 3/25

10/10 5s 549ms/step -
accuracy: 0.3367 - loss: 1.4294 - val_accuracy: 0.4453 - val_loss: 1.3387

Epoch 4/25

10/10 5s 403ms/step -
accuracy: 0.4633 - loss: 1.3648 - val_accuracy: 0.3438 - val_loss: 1.3687

Epoch 5/25

10/10 4s 380ms/step -
accuracy: 0.4500 - loss: 1.2392 - val_accuracy: 0.5547 - val_loss: 1.2220

Epoch 6/25

10/10 5s 514ms/step -
accuracy: 0.5300 - loss: 1.1798 - val_accuracy: 0.4375 - val_loss: 1.2195

Epoch 7/25

10/10 5s 449ms/step -
accuracy: 0.5633 - loss: 1.1517 - val_accuracy: 0.5312 - val_loss: 1.1374

Epoch 8/25

10/10 4s 384ms/step -

accuracy: 0.5933 - loss: 1.0920 - val_accuracy: 0.6016 - val_loss: 1.0801
 Epoch 9/25
 10/10 7s 566ms/step -
 accuracy: 0.6600 - loss: 1.0323 - val_accuracy: 0.6562 - val_loss: 1.0298
 Epoch 10/25
 10/10 9s 372ms/step -
 accuracy: 0.6200 - loss: 1.0237 - val_accuracy: 0.5703 - val_loss: 1.0352
 Epoch 11/25
 10/10 5s 493ms/step -
 accuracy: 0.6900 - loss: 0.9671 - val_accuracy: 0.5547 - val_loss: 0.9938
 Epoch 12/25
 10/10 5s 434ms/step -
 accuracy: 0.6767 - loss: 0.9104 - val_accuracy: 0.6875 - val_loss: 0.9229
 Epoch 13/25
 10/10 4s 368ms/step -
 accuracy: 0.7200 - loss: 0.8567 - val_accuracy: 0.6562 - val_loss: 0.9765
 Epoch 14/25
 10/10 4s 376ms/step -
 accuracy: 0.6933 - loss: 0.8348 - val_accuracy: 0.5781 - val_loss: 0.8910
 Epoch 15/25
 10/10 6s 439ms/step -
 accuracy: 0.7300 - loss: 0.7917 - val_accuracy: 0.6797 - val_loss: 0.8690
 Epoch 16/25
 10/10 4s 384ms/step -
 accuracy: 0.7033 - loss: 0.7772 - val_accuracy: 0.5859 - val_loss: 0.8757
 Epoch 17/25
 10/10 6s 485ms/step -
 accuracy: 0.6700 - loss: 0.7871 - val_accuracy: 0.6328 - val_loss: 0.8235
 Epoch 18/25
 10/10 5s 423ms/step -
 accuracy: 0.7500 - loss: 0.7044 - val_accuracy: 0.5781 - val_loss: 0.9066
 Epoch 19/25
 10/10 4s 380ms/step -
 accuracy: 0.7233 - loss: 0.7354 - val_accuracy: 0.6172 - val_loss: 0.8512
 Epoch 20/25
 10/10 6s 488ms/step -
 accuracy: 0.7033 - loss: 0.7677 - val_accuracy: 0.6562 - val_loss: 0.8438
 Epoch 21/25
 10/10 5s 431ms/step -
 accuracy: 0.7800 - loss: 0.6585 - val_accuracy: 0.6719 - val_loss: 0.7534
 Epoch 22/25
 10/10 4s 386ms/step -
 accuracy: 0.7500 - loss: 0.6361 - val_accuracy: 0.6719 - val_loss: 0.7235
 Epoch 23/25
 10/10 5s 480ms/step -
 accuracy: 0.8067 - loss: 0.6018 - val_accuracy: 0.7109 - val_loss: 0.7300
 Epoch 24/25
 10/10 5s 425ms/step -

```

accuracy: 0.7333 - loss: 0.6149 - val_accuracy: 0.6250 - val_loss: 0.7858
Epoch 25/25
10/10          4s 382ms/step -
accuracy: 0.7667 - loss: 0.5996 - val_accuracy: 0.6250 - val_loss: 0.9038
1/1            0s 179ms/step
Even deeper CNN Test Accuracy: 0.7200

```

2.5.1 Challenge 4 conclusion

The even deeper CNN got 0.72 test accuracy. This was better than the shallow CNN result of 0.36, but it was lower than the original deeper CNN result of 1.00. In this run, adding more layers did not make the model better than the deeper CNN. I think the even deeper model may be harder to train well because the dataset is small, which leads to overfitting problem. So my best model in this run was still the original deeper CNN.

```

[4]: # These are for exporting the notebook as a PDF later if needed
#!apt-get update
#!sudo apt-get install texlive-xetex texlive-fonts-recommended
    ↳ texlive-plain-generic pandoc
!jupyter nbconvert --to pdf /content/drive/MyDrive/BMET5933/WEEK_10/
    ↳ Week_10_Li_Code_file_ipynb_final_version.ipynb

```

```

[NbConvertApp] WARNING | pattern '/content/drive/MyDrive/BMET5933/WEEK_10/Week_1
0_Li_Code_file_ipynb_final_version.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.

```

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y
 Answer yes to any questions instead of prompting.
 Equivalent to: [--JupyterApp.answer_yes=True]

--execute
 Execute the notebook prior to export.
 Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors
 Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, too.
 Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin
 read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
 Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout
 Write notebook output to stdout instead of files.
 Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace
 Run nbconvert in place, overwriting the existing notebook (only relevant when converting to notebook format)
 Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output
 Clear output of current file and save in place, overwriting the existing notebook.
 Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=
 --ClearOutputPreprocessor.enabled=True]

--coalesce-streams
 Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
 Equivalent to: [--NbConvertApp.use_output_suffix=False
 --NbConvertApp.export_format=notebook --FilesWriter.build_directory=
 --CoalesceStreamsPreprocessor.enabled=True]

--no-prompt
 Exclude input and output prompts from converted document.
 Equivalent to: [--TemplateExporter.exclude_input_prompt=True
 --TemplateExporter.exclude_output_prompt=True]

--no-input
 Exclude input cells and output prompts from converted document.
 This mode is ideal for generating code-free reports.
 Equivalent to: [--TemplateExporter.exclude_output_prompt=True
 --TemplateExporter.exclude_input=True
 --TemplateExporter.exclude_input_prompt=True]

--allow-chromium-download
 Whether to allow downloading chromium if no suitable version is found on the system.

Equivalent to: [--WebPDFExporter.allow_chromium_download=True]

--disable-chromium-sandbox

Disable chromium security sandbox when converting to PDF..

Equivalent to: [--WebPDFExporter.disable_sandbox=True]

--show-input

Shows code input. This flag is only useful for dejavu users.

Equivalent to: [--TemplateExporter.exclude_input=False]

--embed-images

Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.

Equivalent to: [--HTMLExporter.embed_images=True]

--sanitize-html

Whether the HTML in Markdown cells and cell outputs should be sanitized..

Equivalent to: [--HTMLExporter.sanitize_html=True]

--log-level=<Enum>

Set the log level by value or name.

Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']

Default: 30

Equivalent to: [--Application.log_level]

--config=<Unicode>

Full path of a config file.

Default: ''

Equivalent to: [--JupyterApp.config_file]

--to=<Unicode>

The export format to be used, either one of the built-in formats

['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']

or a dotted object name that represents the import path for an

``Exporter`` class

Default: ''

Equivalent to: [--NbConvertApp.export_format]

--template=<Unicode>

Name of the template to use

Default: ''

Equivalent to: [--TemplateExporter.template_name]

--template-file=<Unicode>

Name of the template file to use

Default: None

Equivalent to: [--TemplateExporter.template_file]

--theme=<Unicode>

Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)

Default: 'light'

Equivalent to: [--HTMLExporter.theme]

--sanitize_html=<Bool>

Whether the HTML in Markdown cells and cell outputs should be sanitized.This should be set to True by nbviewer or similar tools.

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
```

```
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.